

微信格斗小游戏 Animancer 全功能迁移策划案

源项目 fighting-wx 迁移至 AwCon 框架

文档目的：定义功能边界、技术架构、实施阶段、验收标准与风险控制

目标框架：AwCon - Unity 2022.3.62 - Animancer 驱动

源项目：fighting-wx - Unity 2022.3.14 - 传统 Animator 状态机

版本：v1.0

日期：2026 年 9 月 19 日

1 执行摘要

本方案以“玩法与内容等价、动画架构重构”为总原则，将 F:\.Untiy\YY\fighting-wx 的微信横版格斗小游戏完整迁移到 AwCon 框架。迁移不复制原 Animator Controller、Animator Override Controller、Trigger、Bool、Float 或 StateMachineBehaviour；保留动画资源、角色数据、关卡规则、战斗节奏和平台能力，并以 Animancer、代码状态机、动作仲裁器和数据化时间线替代原有控制器。

核心结论：目标框架已经具备 AnimancerFacade、全身状态机、动作优先级仲裁、分层动画、IK、装备、输入管线和伤害仲裁。迁移的主要工作不是“替换播放 API”，而是建立一套格斗战斗域：以 FightIntent 表达输入和 AI 意图，以 FightStateMachine 决定权威状态，以 CombatTimeline 在精确帧点触发命中、音效、特效、拾取和抛投事件。

建议先完成单角色对单敌人的垂直切片：移动、拳脚、命中、受击、击倒、死亡。确认动画时序、优先级和物理模型后，再批量迁移连段、武器、抓取、关卡、菜单和微信平台层。

2 已确认的项目基线

项目	确认结果	迁移影响
源项目版本	Unity 2022.3.14f1c1	升级至目标 2022.3.62 的版本跨度处于同一 LTS 代际，资源导入仍需分批验证。
目标项目版本	Unity 2022.3.62f3c1	以目标项目 Package 与项目设置为唯一运行基线。
源场景	SplashScene、Menu、Level，共 3 个启用场景	按启动、元游戏、关卡战斗三段式迁移。
源内容规模	119 个 Prefab、176 个 .anim、407 个 .asset、5 个 FBX	需建立资产清单和 GUID 可追踪的映射表，不做一次性全量拖拽。
源核心代码	CharacterControl、PlayerControl、EnemyControl	拆解为战斗域、玩家输入适配器与敌人决策模块。
目标动画能力	AnimancerFacade 支持 Clip、层、Mask、时间点回调、结束回调、全身动作	可完全替代角色 Animator 状态机。
目标角色基础	CharacterController、MotionDriver、Health Arbiter、ActionArbiter、装备与 IK	格斗模块应在这些既有边界内扩展，而非重写角色总控。

3 迁移范围与交付边界

本项目以“可在目标项目中从启动到完成一局战斗，并在微信环境完成对应平台行为”为完整交付边界。下表列出全部需要迁移的功能域。

功能域	原项目能力	目标交付
角色基础	待机、走跑、转向、跳跃、落地、舞台边界	格斗角色控制器与动画状态均由 AwCon 状态机驱动。
近战战斗	拳、脚、前后方向攻击、空中攻击、地面攻击、连段	数据化招式、输入缓冲、命中盒、连段窗口与取消规则。
防守与受击	防御、闪避、硬直、击退、击倒、起身、死亡	伤害请求、受击仲裁、无敌帧、霸体和死亡锁定。

功能域	原项目能力	目标交付
抓取交互	抓取、被抓、抓取连段、挣脱、地面抓取	双角色同步状态、对位锚点、共享事件时间线。
武器与场景物	拾取、持握、攻击、投掷、损坏、搬起、抛掷、破坏物	扩展现有装备系统和独立交互物模块。
敌人与 Boss	追击、距离控制、随机攻击、格挡、闪避、跳攻、拾取武器	基于 Intent 的战斗 AI 与配置化 Boss 特殊行为。
关卡与表现	波次、敌人出生、胜负、血条、连击字、音效、特效、慢动作、震屏、镜头	FightSession 事件总线与 HUD/Presentation 层。
元游戏	启动、菜单、商店、升级、角色选择/解锁、存档	独立 MetaGame 模块，不与战斗状态相互依赖。
微信平台	广告、分享、排行榜、录屏、数据持久化	PlatformService 接口、微信实现与编辑器 Mock。

4 目标架构

目标架构采用“意图、仲裁、状态、表现”四层分离。AI 与玩家输入只能提交意图；只有状态机可改变角色权威状态；Animancer 只负责表现 Clip 与回调；伤害、特效、音效、镜头和 UI 均订阅战斗事件。

层级	职责	禁止事项
输入与 AI	产生 Move、Attack、Defence、Grab、Interact 等 FightIntent。	不得直接播放 Clip、修改 Animator 参数或直接结算伤害。
仲裁层	根据优先级、可打断性、硬直、无敌、资源与目标条件接受/拒绝请求。	不得依赖动画控制器当前状态名称。
状态层	维护移动、攻击、抓取、受击、倒地、死亡等权威游戏状态。	不得由表现层反向篡改状态。
表现层	Animancer 播放、时间线回调、VFX、SFX、相机、HUD。	不得承担游戏规则或网络/存档权威。

运行链路：玩家输入或 AI 决策 -> FightIntent -> FightArbiter -> FightStateMachine -> AnimancerFacade -> CombatTimeline -> Hitbox、伤害、音效、特效、镜头和 UI。

5 Animator 状态机到 Animancer 的转换规范

源项目通过 AnimatorOverrideController 在运行时替换 Idle、Walk、Attack、Reaction、Die 等槽位，并用 Trigger/Bool/Float 推动过渡。目标项目改为“每个动作直接引用 Clip”，所有时序由动作配置驱动。Animator 组件仍保留 Avatar、骨骼和 Humanoid 功能，但 Runtime Animator Controller 必须为空。

原 Animator 机制	Animancer 替代方式	实现要点
Speed Float	Locomotion ClipTransition 或 MixerState 参数	移动状态持续喂入输入速度；不通过 Animator.SetFloat。
Attack/Reaction/Die Trigger	ActionRequest + FightState	以动作优先级和可打断规则取代 Trigger 竞争。
Jump/Defence Bool	明确的 JumpState/DefenceState	状态进入和退出承担逻辑，不保留布尔同步。
AnimatorOverrideController	CombatMoveSO 直接引用 AnimationClip	角色差异、武器差异通过 FighterDefinitionSO 配置。
StateMachineBehaviour	State Enter/Exit 与 CombatTimeline	移除 JumpState、KnockDownState、PoweringUpScr 等 Animator 回调。

原 Animator 机制	Animator 替代方式	实现要点
AnimationEvent 组件函数	CombatTimelineEvent 或 Animator AddCallback	关键帧可审计、可复制、可数据化调整。

6 核心数据设计

6.1 CombatMoveS0

- 基础播放：AnimationClip、FadeDuration、PlaybackSpeed、Mirror、Loop、Layer、AvatarMask。
- 战斗属性：优先级、可打断规则、受击抗性、是否允许重力、移动模式、转向策略。
- 时间线：按 normalizedTime 排列的命中盒开关、连段窗口、伤害结算、音效、特效、镜头、状态完成事件。

6.2 FighterDefinitionS0

- 角色基础属性、移动数据、拳脚/背身/空中/地面攻击组、防御、闪避、抓取、受击、倒地、死亡和 PowerUp 配置。
- 与源 AttackSet、ReactionSet、GrabSet 的字段一一映射；原 Animation0 转为 Clip、播放速度、镜像三元配置。

6.3 DamageRequest 与 ReactionProfileS0

- DamageRequest 至少包含攻击者、攻击方向、伤害、命中部位、轻重、击退、击倒、武器、无敌帧和慢动作意图。
- ReactionProfileS0 按方向、命中部位、攻击等级、空中/倒地状态选择受击动作，避免 CharacterControl 中的大量条件分支。

7 战斗状态与优先级

优先级	状态组	典型状态	可被打断规则
100+	终止状态	Death	除重生/关卡重置外不可打断。
90	失控状态	Knockdown、Airborne、Grabbed	仅指定地面追击或系统流程可改变。
80	高承诺动作	Grab、Throw、Lift、PowerUp	受击按招式规则打断；移动不可打断。
60	战斗动作	Attack、JumpAttack、Reaction、Defence	受击、死亡、特定取消窗口可覆盖。
40	机动力动作	Jump、Dodge、Land	高优先级受击与攻击请求按规则覆盖。
0	基础状态	Idle、Move	任意有效战斗请求可进入。

第一阶段可复用现有 ActionArbiter 与 OverrideState 承接全身动作；正式战斗版应增加 FightState 基类和战斗专用仲裁器，使攻击结束、命中、取消和回归移动不再依赖通用 OverrideState 的单一结束回调。

8 动画时间线事件清单

事件	原项目来源	新行为
HitboxOn / HitboxOff	Check4Hit 协程与攻击 Clip 事件	按数据启用盒体并对同一目标去重。
ComboOpen / ComboClose	CancelCombo 与攻击锁	决定输入缓冲何时可衔接下一招。
AttackUnlock	AttackLock	解除动作承诺并恢复转向/移动权限。
PickWeapon / ThrowWeapon	AnimationEvents	在指定帧执行附着、脱手、刚体与伤害初始化。
GrabHit / GrabReaction	AnimationEvents	同步攻击者与被抓者的伤害与受击播放。

事件	原项目来源	新行为
Lift / Release	Lifting、Lifted、LiftThrown	切换物体父级、物理状态和投掷速度。
Sfx / Vfx / Camera	PlaySFX、HitEffect、Shake、SlowMotion	只发送表现事件，不改变战斗规则。
StateComplete	Toldle 与状态机行为	由状态机选择回到移动、空中或后续战斗状态。

9 模块拆分与文件建议

模块	建议职责	主要依赖
Fight Core	FightIntent、FightStateMachine、FightArbiter、状态基类	BBBCharacterController、RuntimeData、AnimatorFacade
Fight Data	CombatMoveSO、FighterDefinitionSO、ReactionProfileSO、Timeline	Unity ScriptableObject、AnimationClip
Combat Resolver	Hitbox、Damage、Reaction、Invulnerability、Knockback	HealthArbiter、Physics、MotionDriver
Interaction	WeaponCombat、Pickup、Throw、Grapple、LiftableObject	Inventory、IK、Rigidbody
AI	FightEnemyBrain、目标感知、距离控制、Boss行为	FightIntent、FighterDefinitionSO
Presentation	CombatPresentation、HUD Presenter、Camera、Audio、VFX	事件总线、Cinemachine、AudioDriver
Session	FightSession、Wave、Spawn、Win/Lose、关卡边界	场景、对象池、HUD
Platform	PlatformService、WeChatPlatformService、EditorMock	存档、广告、分享、排行榜

10 分阶段实施计划

阶段	目标与工作内容	完成判定
0 基线冻结	建立源资产清单、动作映射表、场景与输入行为录像；确认每个 Clip 的循环、镜像、根位移与关键帧。	可追踪的资产表；源项目可作为逐项对照基准。
1 垂直切片	实现 FighterDefinitionSO、CombatMoveSO、FightIntent、单敌人 AI、移动、单拳、命中、受击、击倒、死亡。	角色 Animator Controller 为空；核心一对一战斗完整可玩。
2 基础战斗完成	实现拳脚连段、方向攻击、跳跃/空中攻击、格挡、闪避、起身、输入缓冲和取消规则。	全部基础动作按时间线结算，无 Animator 参数调用。
3 交互与武器	实现武器拾取、装备、武器连段、投掷、损坏、可破坏物、搬起/投掷物和抓取双人同步。	可复现源 Level 的所有近场交互。
4 敌人与关卡	实现敌人行为、Boss、波次、出生、胜负、血条、镜头、慢动作、特效和音效。	完整关卡可从开始打到结算。
5 元游戏与平台	迁移启动、菜单、商店、升级、角色解锁、存档和微信平台服务。	Splash -> Menu -> Level 流程可运行；编辑器 Mock 可测试。
6 验收与优化	自动/半自动回归、性能分析、设备测试、微信真机验证、Animator 依赖清理。	达到验收矩阵，完成可发布候选版本。

11 资产迁移流程

- ☐ 只复制源 Assets 中明确需要的模型、动画、材质、贴图、音效、Prefab 和 ScriptableObject；不复制 Library、Temp、Logs、obj。
- ☐ 每批资源先导入隔离目录，核对 Humanoid Avatar、动画导入设置、循环、Root Transform、事件、材质与 Shader。
- ☐ 不手改 .unity、.prefab、.asset YAML。场景、Prefab 与 ScriptableObject 的结构调整只通过 Unity Editor/MCP 或 Unity API 完成。
- ☐ 为每个源 AttackSet、ReactionSet、GrabSet、Weapons 资产生成映射记录：源路径、目标 SO、Clip、事件帧、验证状态、负责人。

12 物理与位移策略

- 源项目以 Rigidbody 和 Translate 混合驱动角色；目标框架以 CharacterController 与 MotionDriver 为基础。新方案规定角色本体只保留一种位移权威：默认使用 CharacterController。攻击推进、击退、跳跃和动画位移由 MotionDriver 或动作曲线驱动；投掷武器、可破坏物和被抛出物体使用 Rigidbody。
- ☐ 不得在同一角色上同时让 Rigidbody 与 CharacterController 争夺位置。
 - ☐ 攻击 Clip 的 Root Motion 必须逐条审查：静态招式关闭；位移动作可提取曲线或在限定状态下由 Animancer 全身动作启用。
 - ☐ 命中检测使用 NonAlloc Physics 查询和攻击实例的已命中集合，保证同一窗口不重复伤害同一碰撞体。

13 微信平台与外部依赖策略

广告、分享、排行榜、录屏、网络请求和存档不得继续散落在 UI 与战斗脚本中。定义 PlatformService 接口；编辑器和 Windows 环境使用 EditorMockPlatformService，微信构建使用 WeChatPlatformService。战斗模块只能订阅平台结果，不直接调用 SDK。

能力	接口示例	编辑器行为
激励广告	ShowRewardedAd(callback)	立即返回模拟成功/失败，并允许测试两条分支。
分享与录屏	Share(payload)、StartRecording、StopRecording	记录调用参数，不调用真实平台。
排行榜与好友	LoadRank、LoadFriendRank	返回本地测试数据或空数据。
存档	LoadSave、Save	使用独立本地存储，微信实现换为平台存储。

14 测试与验收矩阵

类别	必须验证的场景	通过标准
动画架构	角色启动、移动、攻击、受击、死亡	角色 Runtime Animator Controller 为空；无 SetTrigger/SetBool/SetFloat/OverrideController 运行依赖。
招式时序	每一招的起手、命中、连段、收招	关键帧误差可数据调整；不早结算、不漏结算、不重复结算。
状态优先级	攻击中受击、倒地中受击、死亡、抓取、闪避	低优先级状态不能越权覆盖高优先级状态。
双人交互	抓取、挣脱、武器投掷命中、地面攻击	双方动作、位置、伤害与恢复时机同步。
关卡流程	启动、菜单、选角、关卡、胜利、失败、重开	三个原始主流程在目标项目中可连续走通。
平台隔离	编辑器运行、微信运行	编辑器不缺 SDK；微信平台调用可追踪且失败可回退。
性能	多敌人、特效、命中扫描、对象池复用	无逐帧 GC 高峰；Physics 查询使用 NonAlloc；对象池回收后无动画回调残留。

15 主要风险与缓解措施

风险	影响	缓解措施
一次性导入全部资产	问题定位困难，Prefab 和 GUID 引用容易失效。	按角色、动画、交互物、场景分批迁移，每批完成后运行验证。
把 Animator 参数直接翻译为 API 调用	保留隐式状态，后续难以维护。	先定义状态机与仲裁规则，再接 Clip 和时间线。
Root Motion 与 CharacterController 冲突	滑步、穿模、击退方向错误。	逐 Clip 标注移动模式，只保留一个位置权威。
抓取动画没有双人同步设计	两角色错位、提前结算或无法释放。	使用共享 GrappleSession 和同一归一化时间基准。
微信 SDK 侵入核心逻辑	编辑器无法测试，发布排查困难。	强制 PlatformService 依赖倒置和 Mock。
源项目隐藏依赖未发现	迁移后出现缺资源或功能缺口。	阶段 0 建立场景依赖清单与行为录像，验收按清单逐项关闭。

16 首个迭代的可执行任务清单

- ☐ 创建 Fight Core、Fight Data、Combat Resolver、Interaction、AI、Presentation、Session、Platform 的目录与程序集边界。
- ☐ 制作一个 FighterDefinitionSO、一个 CombatMoveSO 和一个 ReactionProfileSO，引用一名角色的 Idle、Move、Punch、Hit、Knockdown、Death Clip。
- ☐ 实现 AttackState 与 HitReactionState，使用 AnimancerFacade 的全身播放、AddCallback 与结束回调。
- ☐ 将命中盒、伤害、击退和受击 Clip 选择接入 HealthArbiter，不直接操作 Animator。
- ☐ 建立源项目与目标项目的一对一录屏验证用例：待机、移动、单拳、受击、击倒、死亡。
- ☐ 通过后再开始批量转换 AttackSet、ReactionSet、GrabSet 和 Weapons 数据。

17 结论与决策

本迁移应按“先战斗内核、后内容批量化、最后平台层”的顺序推进。目标框架的 AnimancerFacade 与状态机能力已经覆盖源 Animator Controller 的角色动画需求；真正决定质量的是战斗状态权威、动作优先级、关键帧时间线和物理位移的重新定义。

项目启动后，第一项交付应是单角色对单敌人的可玩垂直切片。该切片通过验收后，所有剩余角色、招式、武器、抓取和关卡内容都将变成可配置的数据迁移工作，而不是继续扩张 Animator 状态机。

附录 A 源项目到目标模块映射

源脚本/资产	目标模块	处理方式
CharacterControl	Fight Core + Combat Resolver + Interaction	拆分移动、攻击、受击、武器、抓取、搬运职责；禁止原样复制。
PlayerControl	FightInputAdapter	将 ControlFreak 输入翻译到 AwCon InputData 与 Intent。
EnemyControl	FightEnemyBrain	保留决策参数，输出 Intent，不直接调用动画。
Attack0 / AttackSet	CombatMoveS0 / ComboDefinitionS0	逐个动作映射 Clip、命中盒、时序、优先级。
ReactionSet	ReactionProfileS0	按受击条件选择反应动作。
GrabSet / GrabAnimations	GrappleDefinitionS0	建立攻击者/被抓者同步动作和释放规则。
Weapons	FightWeaponDefinitionS0 + Runtime Weapon	对接现有 Inventory 与 EquipmentDriver。
AnimationEvents	CombatTimelineDispatcher	迁移为代码注册的时间线事件。
Animator Controller 与 StateMachineBehaviour	无	不迁移；以 C# State 与 Animancer 回调替换。